

**UJIAN TENGAH SEMESTER
MATA KULIAH INTEROPERABILITAS**



**Dosen Pengampu :
Wiktasari, S.T., M.Kom.**

**Disusun Oleh :
MUHAMMAD RAAFI HARIYADI 4.33.25.8.03**

**PORGRAM STUDI TEKNIK REKAYASA KOMPUTER
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI SEMARANG
2026**

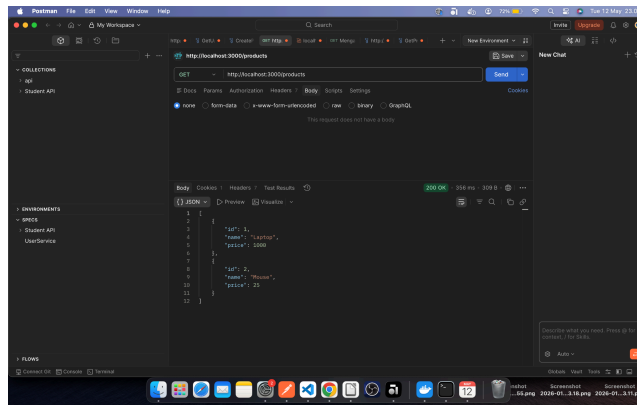
1. Restful API

a. Source code

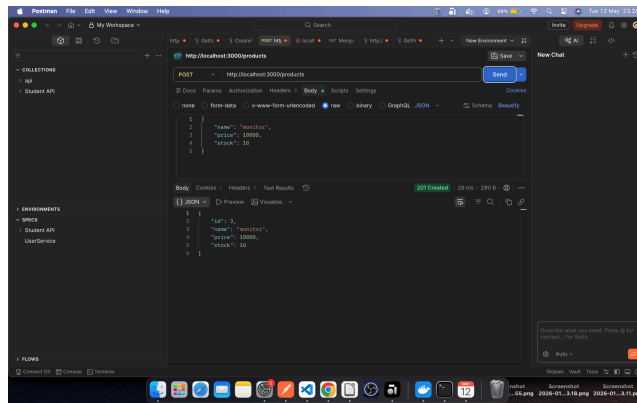
```
1  const express = require("express");
2
3  const app = express();
4  const PORT = 3000;
5
6  // Middleware
7  app.use(express.json());
8
9  // Dummy database
10
11  let products = [
12    { id: 1, name: "Laptop", price: 1000 },
13    { id: 2, name: "Mouse", price: 25 }
14  ];
15
16  // GET /products - semua produk
17  app.get("/products", (req, res) => {
18    res.setHeader("Content-Type", "application/json");
19    res.status(200).json(products);
20  });
21
22  // GET /products/:id - detail produk
23  app.get("/products/:id", (req, res) => {
24    const id = parseInt(req.params.id);
25    const product = products.find(p => p.id === id);
26
27    if (!product) {
28      return res.status(404).json({ error: "Product not found" });
29    }
30
31    res.setHeader("Content-Type", "application/json");
32    res.status(200).json(product);
33  });
34
35  // POST /products - tambah produk
36  app.post("/products", (req, res) => {
37    const { name, price } = req.body;
38
39    if (!name || price === null) {
40      return res.status(400).json({ error: "Name and price are required" });
41    }
42
43    const newProduct = {
44      id: products.length ? products[products.length - 1].id + 1 : 1,
45      name,
46      price
47    };
48
49    products.push(newProduct);
50
51    res.setHeader("Content-Type", "application/json");
52    res.status(201).json(newProduct);
53  });
54
55  // PUT /products/:id - update produk
56  app.put("/products/:id", (req, res) => {
57    const id = parseInt(req.params.id);
58    const product = products.find(p => p.id === id);
59
60    if (!product) {
61      return res.status(404).json({ error: "Product not found" });
62    }
63
64    const { name, price } = req.body;
65
66    if (name) product.name = name;
67    if (price !== null) product.price = price;
68    res.setHeader("Content-Type", "application/json");
69    res.status(200).json(product);
70  });
71
72  // DELETE /products/:id - hapus produk
73  app.delete("/products/:id", (req, res) => {
74    const id = parseInt(req.params.id);
75    const index = products.findIndex(p => p.id === id);
76
77    if (index === -1) {
78      return res.status(404).json({ error: "Product not found" });
79    }
80
81    const deleted = products.splice(index, 1);
82
83    res.setHeader("Content-Type", "application/json");
84    res.status(200).json({
85      message: "Product deleted",
86      product: deleted[0]
87    });
88  });
89
90  app.listen(PORT, () => {
91    console.log(`Server running on port ${PORT}`);
92  });
```

b. Hasil Uji

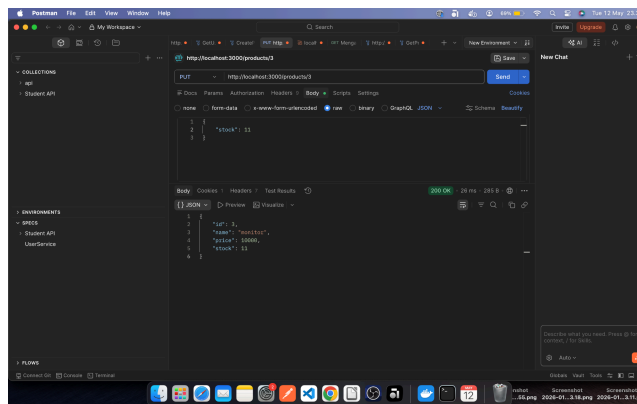
- GET



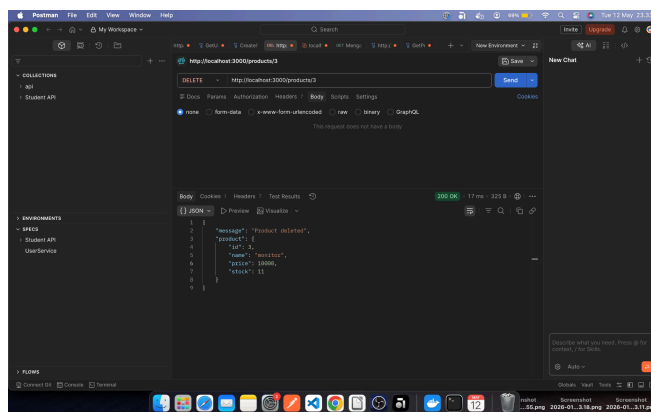
- POST



- PUT



- DELETE



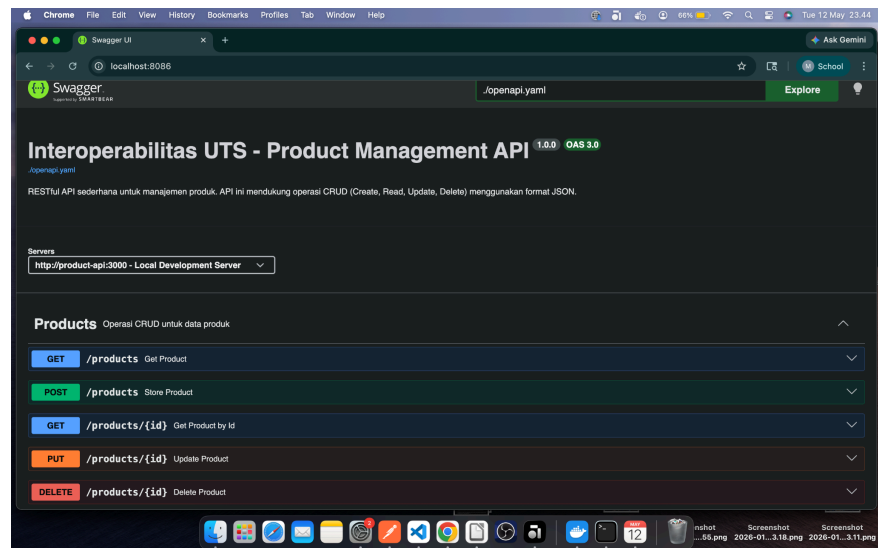
2. Open API

a. file openapi.yaml

```
1  operapi: 3.0.3
2
3  info:
4    title: Intersperabilitas UTS - Product Management API
5    version: 1.0.0
6    description:
7      RESTful API sederhana untuk manajemen produk.
8      API ini mendukung operasi CRUD (Create, Read, Update, Delete)
9      menggunakan format JSON.
10
11  servers:
12    - url: http://product-api:3000
13      description: Local Development Server
14
15  tags:
16    - name: Products
17      description: Operasi CRUD untuk data produk
18
19  paths:
20    /products:
21      get:
22        tags:
23          - Products
24        summary: Get Product
25        description: Menampilkan seluruh data produk.
26        responses:
27          '200':
28            description: Berhasil mengambil daftar produk.
29            content:
30              application/json:
31                schema:
32                  type: array
33                  items:
34                    $ref: '#/components/schemas/Product'
35          '404':
36            description: Produk tidak ditemukan.
37      post:
38        tags:
39          - Products
40        summary: Store Product
41        description: Menambahkan produk baru.
42        requestBody:
43          required: true
44          content:
45            application/json:
46              schema:
47                $ref: '#/components/schemas/ProductInput'
48              example:
49                name: Keyboard
50                price: 250000
51                stock: 15
52          responses:
53            '201':
54              description: Produk berhasil ditambahkan.
55              content:
56                application/json:
57                  schema:
58                    type: object
59                    properties:
60                      type: string
61                      example: Produk berhasil ditambahkan
62                      data:
63                        $ref: '#/components/schemas/Product'
64            '400':
65              description: Data input tidak valid.
66          /products/{id}:
67            get:
68              tags:
69                - Products
70              summary: Get Product by Id
71              description: Menampilkan detail produk berdasarkan ID.
72              parameters:
73                - name: id
74                  in: path
75                  required: true
76                  description: ID produk.
77              schema:
78                type: integer
79                examples: 1
80              responses:
81                '200':
82                  description: Detail produk ditemukan.
83                  content:
84                    application/json:
85                      schema:
86                        $ref: '#/components/schemas/Product'
87                '404':
88                  description: Produk tidak ditemukan.
89          put:
90
```

```
91    put:
92      tags:
93        - Products
94      summary: Update Product
95      description: Memperbarui data produk berdasarkan ID.
96      parameters:
97        - name: id
98          in: path
99          required: true
100          description: ID produk.
101      schema:
102        type: integer
103        examples: 1
104      requestBody:
105        required: true
106        content:
107          application/json:
108            schema:
109              $ref: '#/components/schemas/ProductInput'
110            example:
111              name: Mechanical Keyboard
112              price: 500000
113              stock: 8
114      responses:
115        '200':
116          description: Produk berhasil diperbarui.
117          content:
118            application/json:
119              schema:
120                type: object
121                properties:
122                  message:
123                    type: string
124                    example: Produk berhasil diperbarui
125                  data:
126                    $ref: '#/components/schemas/Product'
127        '404':
128          description: Produk tidak ditemukan.
129      delete:
130        tags:
131          - Products
132        summary: Delete Product
133        description: Menghapus produk berdasarkan ID.
134        parameters:
135          - name: id
136            in: path
137            required: true
138            description: ID produk.
139        schema:
140          type: integer
141          examples: 1
142        responses:
143          '200':
144            description: Produk berhasil dihapus.
145            content:
146              application/json:
147                schema:
148                  type: object
149                  properties:
150                    message:
151                      type: string
152                      example: Produk berhasil dihapus
153                    data:
154                      $ref: '#/components/schemas/Product'
155          '404':
156            description: Produk tidak ditemukan.
157
158  components:
159    schemas:
160      Product:
161        type: object
162        required:
163          - id
164          - name
165          - price
166          - stock
167        properties:
168          id:
169            type: integer
170            examples: 1
171          name:
172            type: string
173            examples: Laptop
174          price:
175            type: number
176            examples: 250000
177          stock:
178            type: integer
179            examples: 10
180      ProductInput:
181        type: object
182        required:
183          - name
184          - price
185          - stock
186        properties:
187          name:
188            type: string
189            examples: Keyboard
190          price:
191            type: number
192            examples: 250000
193          stock:
194            type: integer
195            examples: 15
196
```

b. Tampilan



3. JSON-RPC

a. source code server

```

1  from flask import Flask, request, jsonify
2  app = Flask(__name__)
3  @app.route('/rpc', methods=['POST'])
4
5  # Tugas
6
7  def rpc():
8      data = request.get_json()
9
10     if data["method"] == "sample.add":
11         result = sum(data["params"])
12         return jsonify({
13             "jsonrpc": "2.0",
14             "result": result,
15             "id": data["id"]
16         })
17
18     elif data["method"] == "sample.multiply":
19         result = data["params"][0] * data["params"][1]
20         return jsonify({
21             "jsonrpc": "2.0",
22             "result": result,
23             "id": data["id"]
24         })
25
26     elif data["method"] == "sample.subtract":
27         result = data["params"][0] - data["params"][1]
28         return jsonify({
29             "jsonrpc": "2.0",
30             "result": result,
31             "id": data["id"]
32         })
33
34     else:
35         return jsonify({
36             "jsonrpc": "2.0",
37             "error": "Method not found",
38             "id": data["id"]
39         })
40
41 if __name__ == '__main__':
42     app.run(host="0.0.0.0", port=5000)

```

b. source code client

- python (multiply)

```

1  import time
2  import requests
3
4  url = "http://python-s:5000/rpc"
5
6  print("Client started", flush=True)
7
8  while True:
9      payload = {
10         "jsonrpc": "2.0",
11         "method": "sample.multiply",
12         "params": [5, 31],
13         "id": 1
14     }
15
16
17     try:
18         print("Sending request...", flush=True)
19
20         res = requests.post(url, json=payload)
21
22         print("Status:", res.status_code, flush=True)
23
24         print("Response:", flush=True)
25         print(res.json(), flush=True)
26
27     except Exception as e:
28         print("Error:", e, flush=True)
29
30     time.sleep(5)
31
32

```

- Node (add)

```

1  async function callRpc() {
2      const payload =
3
4      {
5         jsonrpc: "2.0",
6         method: "sample.add",
7         params: [10, 1],
8         id: 2
9     }
10
11
12
13     while (true) {
14         const res = await fetch("http://python-s:5000/rpc", {
15             method: "POST",
16             headers: { "Content-Type": "application/json" },
17             body: JSON.stringify(payload)
18         });
19
20         console.log(await res.json());
21
22         await new Promise(r => setTimeout(r, 5000));
23     }
24 }
25
26 callRpc();

```

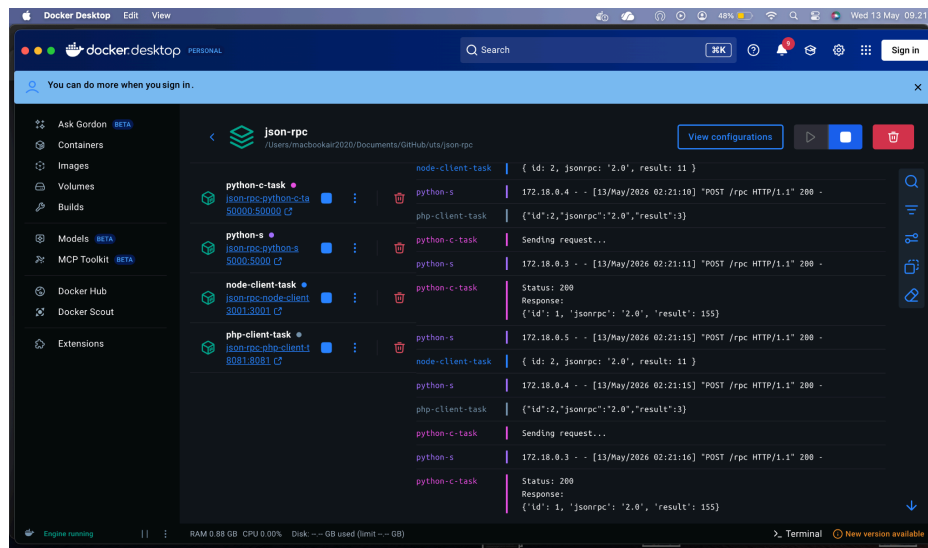
- PHP (subtract)

```

1  <?php
2
3      while (true) {
4          $url = "http://python-s:5000/rpc";
5
6          $data = array(
7              "jsonrpc" => "2.0",
8              "method"  => "sample.subtract",
9              "params"  => array(7, 4),
10             "id"      => 2
11          );
12
13          $options = array(
14              "http" => array(
15                  "header" => "Content-Type: application/json\r\n",
16                  "method" => "POST",
17                  "content" => json_encode($data)
18              )
19          );
20
21          $context = stream_context_create($options);
22          $result = file_get_contents($url, false, $context);
23
24          if ($result === FALSE) {
25              echo "Error: Unable to connect to the RPC server.";
26          } else {
27              echo $result;
28          }
29
30          sleep(5);
31      }
32
33  ?>

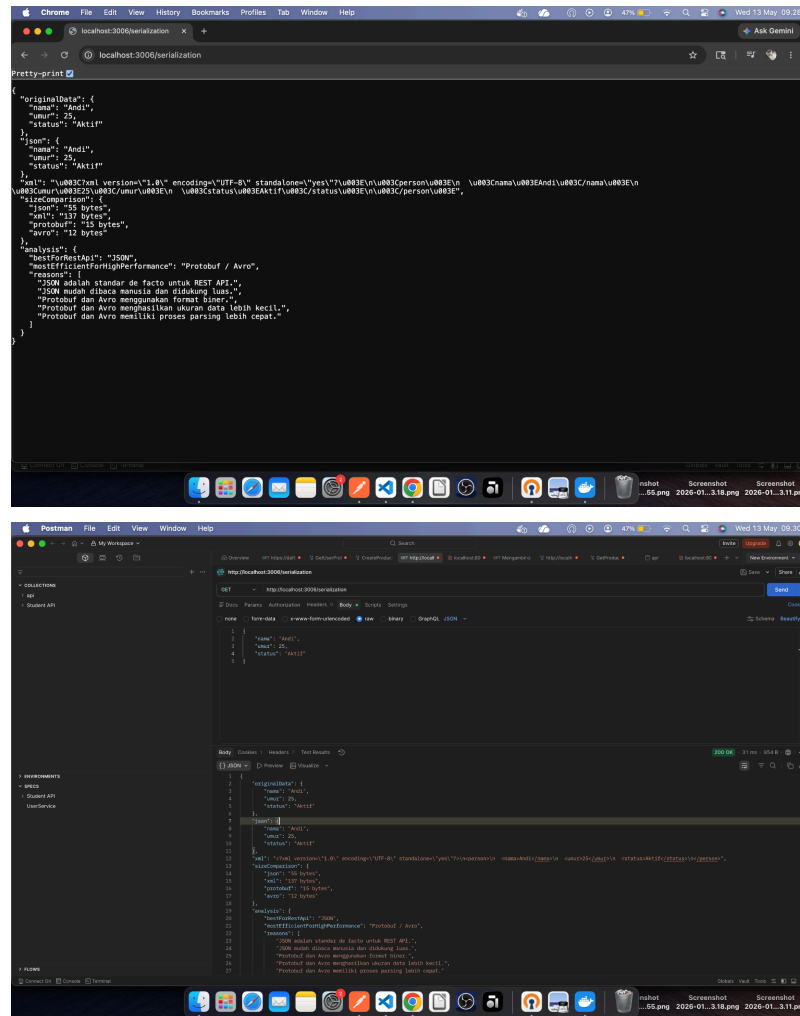
```

c. Hasil eksekusi



4. Data Serialization :

a. Hasil konversi JSON dan XML



b. Analisis perbandingan

Jika Outpunya berupa Json, maka akan menampilkan response berupa format data json pada hasil request baik browser maupun postman, namun jika respon berupa XML maka akan terbaca lebih baik di postman sementara di browser ada sedikit perubahan pada karakter yang tampilkan karena pembacaan hasil request di broswer yang berbeda, jika json berupa data yang terstruktur, dan xml menampilkan tag seperti html

5. Microservices and Event-Driven Architecture

a. servis yang dijalankan

1. User Service

untuk dapat mengakses layanan perlunya data credential dimana user harus memiliki data pada sistem sebagai pengguna(end user).

2. Order Service

untuk melakukan post data harus memiliki service spesifik, Order service ditujukan sebagai event untuk trigger. dalam hal ini sebagai request order yang nantinya dikirim pada message broker sebagai bantuan untuk memproses dari order ke payment.

3. Payment Service

ditujukan untuk menyelesaikan request dari order-service, menerima event dan menampilkan response hasil event.

b. Alur komunikasi antar service

User-service sebagai entitas atau pengguna untuk memberi id agar order-service dapat berjalan.

Order-service melakukan interaksi dengan broker message dengan data yang dikirim sebagai event.

Payment service akan menerima dan merespon event yang dikirimkan dan menyelesaikannya.

c. Event yang digunakan

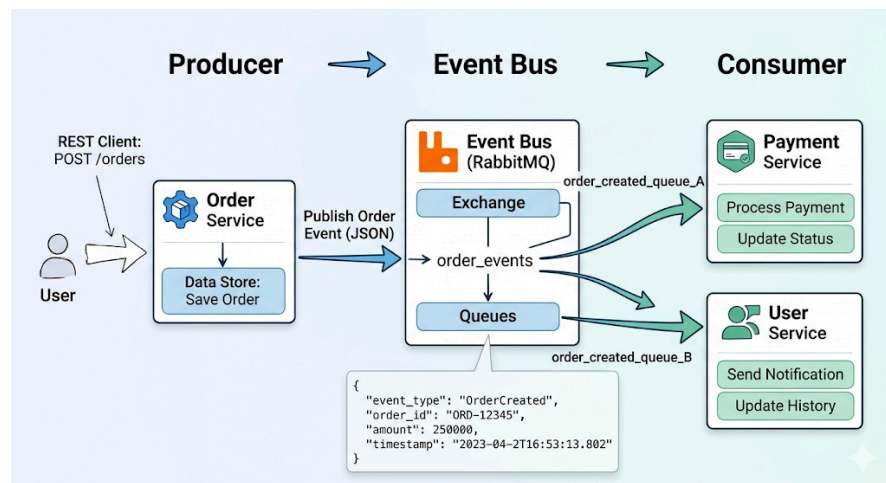
Event yang digunakan adalah POST /orders, dimana /orders yang akan melakukan hit API dengan metode post berisi data JSON dengan format :

```
{
  id: orders.length + 1,
  userId: req.body.userId,
  product: req.body.product
}
```

dan akan diterima di message broker yang kemudian akan diterima oleh payment service.

d. Gambaran alur kerja

Producer -> Event Bus -> Consumer



- e. Komunikasi Rest API dan Message Broker Rabbit.mq
Rest API berperan dalam mengirimkan data dari /orders melalui method POST yang kemudian diterima oleh Message broker yakni rabbit.mq, kemudian rabbit mq akan meneruskan pada payment-service.